

Week 15 Mini-Project · Spec & De-risk

Turn a vague AI ask into an engineering contract, then prove the riskiest brick holds weight.

Theme	Product thinking + system design for an agentic AI system. Define what "good" means in numbers, then test your scariest assumption with a small spike.
Effort	About 1 day for Core. Up to 1.5 days for Advanced. Light on code by design.
New tool	Cloudairy (AI docs + AI architecture / sequence diagrams). Free tier. cloudairy.com
Also use	Python, your chosen LLM API, Chroma, Pydantic (typed data contracts). Optional explore: Instructor for structured LLM output.
You ship	PRD · technical design doc · architecture diagram · risk register · one de-risk spike · one-page executive memo · a GitHub repo.
Interview signal	The ML / AI system-design round: "design X", "what are your KPIs", "what is your riskiest assumption".

How this connects to your capstone

This mini-project **is** the front end of your capstone. Everything you write here becomes your capstone scoping sprint: the PRD, the architecture, the success metrics, and proof that the hardest part is feasible. Next week you build the slice you spec here and measure it against the exact targets you set today. Do this well and your capstone starts with a finished plan instead of a blank page.

The scenario

Northwind is a mid-size B2B SaaS company. Its support agents answer the same product and policy questions all day by digging through scattered help docs, changelogs, and policy PDFs. Handle time is high, answers are inconsistent, and new agents take weeks to ramp. Leadership wants a **Support Copilot**: an agent that retrieves the right document, answers with citations, and can take one bounded action (draft a ticket reply, or look up an order through a tool). They have been burned before by a demo that looked great and then hallucinated in front of a customer. Your mandate: ship something **measurably trustworthy**, not a flashy demo.

You are the founding AI engineer on this pod. You own the spec.

Use your own capstone instead if you prefer. Swap Northwind for your capstone domain and keep every deliverable and target identical. The point is to scope a real agentic system; Northwind is the default vehicle so you are never blocked on ideas.

Your data

Pick one source and assemble **15 to 30 real documents**:

- Your capstone corpus (preferred), or
- A real open-source product's documentation site (export pages to markdown / PDF), or
- A public regulatory / policy PDF set (for example a data-protection or compliance corpus), or
- Domain papers from arXiv (abstracts + intros) if your capstone is research-flavored.

Real and slightly messy is the requirement. No toy datasets. You will reuse this exact corpus next week.

Core mission (Tier C)

1. PRD (1 to 2 pages, written in Cloudairy AI Docs)

- Problem statement: who hurts, and the current cost (handle time, ramp time, inconsistency).
- Users and the top three user stories.
- Scope: what is in v1, and what is explicitly out.
- Functional requirements (what the system does) and non-functional requirements (latency target, cost ceiling per query, data privacy).
- **Success metrics / KPIs.** This is the heart of the week. Define four to six measurable KPIs, each with a definition, how you will measure it, a target, and a "do-not-ship" floor. At minimum cover: groundedness / faithfulness, answer relevance, retrieval hit-rate (did we fetch the document that holds the answer), p95 latency, cost per resolved query, and one business KPI (for example projected handle-time reduction).
- Risks and assumptions.

2. Technical design doc (1 to 2 pages)

- Architecture: ingestion → chunking → embedding → vector store (Chroma) → retriever → prompt assembly → LLM → answer with citations → one bounded action.
- **Data contracts.** Define the typed input and output of each component as Pydantic models and paste them. Engineers reason in contracts, not vibes.
- Model and embedding choices, one line of justification each (default embedding: text-embedding-3-small).
- The one bounded action your agent can take (draft a ticket reply, or call a mock order-lookup tool).

3. Diagrams (Cloudairy)

Draw the architecture diagram and a request sequence diagram (user question → retrieval → generation → action). Export as PNG and commit to `/diagrams`.

4. Risk register

A table of your top five risks, each with likelihood, impact, and a mitigation. Mark the single **highest "riskiest assumption"**; that is what your spike tests.

5. De-risk spike (light code, about 2 to 3 hours)

Write a small Python script that tests **only** your riskiest assumption. For most teams that is retrieval: load your 15 to 30 docs, chunk and embed into Chroma, run 8 to 10 real questions, and check whether the correct chunk appears in the top-k results. Output a small table: question → retrieved-chunk-source → hit (Y/N). You are not building the product; you are proving the scariest brick holds weight. Commit the script and a short `FINDINGS.md`.

Depth ladder

Pick the depth that matches where you are. Higher tiers include everything below them. Anyone who finishes their tier can attempt the Bonus.

Tier C • Core. PRD + design doc + architecture diagram + risk register + spike on the one riskiest assumption + a hit-rate observation in `FINDINGS.md`.

Tier B • Stretch. Add a Cloudairy sequence diagram; a "metric tree" linking each technical metric to the business KPI; and a spike that compares two chunking strategies (for example fixed 512-token vs recursive / semantic) on the same 10 questions, with a reasoned verdict on which retrieves better.

Tier A • Advanced. Add a non-functional plan (cost model per 1,000 queries; a PII / privacy handling note; a failure-mode table covering retrieval miss, out-of-scope question, and tool failure); a deployment and re-index sketch (where it runs, how a new document gets indexed); and a north-star metric paired with a guardrail metric.

Bonus. Use Cloudairy AI to generate a first-draft diagram from your written design, then critique what it got wrong and fix it. Document the before and after.

What "done" looks like (Tier C)

A reviewer opens your repo and within 10 minutes understands what you are building, what "good" means in numbers, the riskiest assumption, and sees evidence (a small table) that retrieval finds the right document for most of your test questions. The PRD names KPIs with targets and do-not-ship floors. The spike script runs end to end on your data.

LLM-integrated task (pass / fail gate)

Use an LLM as a "skeptical staff engineer" to red-team your PRD. Paste your KPI section and ask it to find weak or unmeasurable metrics and missing failure modes. In your repo, paste four things: the prompt, the model's response, **your** critique of that response (where it was right, where it was generic or wrong), and your revised KPI section. A pasted chat with no critique fails the gate. Failing the gate caps your score at Tier C until you resubmit.

Reflection (answer in README, 3 to 5 sentences each)

- Which KPI was hardest to make measurable, and why?
- What is your riskiest assumption, and did the spike raise or lower your confidence?
- If you had to cut scope to ship in half the time, what is the first thing that goes, and why?

Executive memo (one page, interview-ready)

Frame the work as a case study: the problem, your proposed system in three sentences, the KPIs that define success, the riskiest assumption and your evidence, and what you would build first. This is the artifact you walk an interviewer through in a system-design round. Save it as `EXEC_MEMO.md`.

Submission

Push to your GitHub repo (for example `northwind-support-copilot`, or your capstone repo). Suggested structure:

```
/prd /design /diagrams /spike FINDINGS.md EXEC_MEMO.md README.md
```

Submit the repo link through the LMS. You may be picked for a 5-minute viva within 48 hours; you must be able to explain every KPI and every line of your spike script.

Tool pointers (learn fast)

Cloudairy: sign up free, use AI Docs for the PRD and AI Diagram for the architecture and sequence views; both generate from a text prompt and stay editable. **Pydantic:** `pip install pydantic`; define a model per component to lock your data contracts. **Chroma:** you already know it from the RAG module; reuse your pattern.